

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

INTRO TO CRITICAL INFRASTRUCTURE FOR IT LEARNERS

LAB 03 — BUILDING YOUR FIRST PLC PROGRAM

Student Edition

Name: Sidra Shaikh | Registration No: C11/26/ACIS/17349 | Date: 20 September 2026

Authorized ICDFA training environments only
icdfa.edu.ng

Lab 03: Building Your First PLC Program

Course: Intro to Critical Infrastructure for IT Learners | **Estimated Duration:** 4–5 hours

Submission deadline: As published in ICDFA E-Campus.

Authorization & Safety: Perform all activities only on the ICDFA-provided virtual machines, packet captures and isolated training networks. Do not bridge these VMs to production networks, scan external systems, or reuse credentials observed in training data.

Learning Outcomes

- Create a new OpenPLC Ladder Diagram project.
- Define Boolean and Real variables for an industrial tank-fill process.
- Build logic to automatically fill a 0–10 ft tank between operating setpoints.
- Implement high/low alarm logic and an emergency-stop condition.
- Document and validate the intended control behavior before connecting to an HMI.

Laboratory Environment

System	Expected Address	Account	Password
--------	------------------	---------	----------

Note: If the instructor-provided VM image uses updated addressing or credentials, use the values published in E-Campus. Do not alter addressing unless instructed.

Variable Plan

Variable	Data Type	Purpose
TankFillAuto_Cmd	BOOL	Internal automatic fill latch/command
TankValveOpen	BOOL	Discrete command/status for the fill valve
Estop_Cmd	BOOL	Emergency-stop input; TRUE forces automatic fill to stop
TankLevelHi_Alarm	BOOL	High-level alarm state
TankLevelLo_Alarm	BOOL	Low-level alarm state
TankLevel_LIT	REAL	Measured tank level in feet
TankEmpty_SP	REAL	Automatic fill start setpoint (2.0 ft)
TankFull_SP	REAL	Automatic fill stop setpoint (8.0 ft)
TankLoAlarm_SP	REAL	Low-level alarm setpoint (1.0 ft)
TankHiAlarm_SP	REAL	High-level alarm setpoint (9.0 ft)

Pre-Lab Checklist

- Read the complete lab before beginning.
- Confirm you can access the required VM(s) and that virtualization support is enabled.
- Use the instructor-specified isolated/host-only network.
- Create or restore the clean starting snapshot for this lab.
- Prepare a report document with your name, registration number, course, lab number and date.
- Capture evidence as you progress rather than repeating the lab at the end.

Procedure

Step 1. Start the PLC VM from a clean snapshot. Launch OpenPLC Editor and create a new project folder named TankFill.

Step 2. Create a new POU named TankFill and select LD (Ladder Diagram) as the language.

Step 3. Add the variables listed in the Variable Plan table in this manual. Use BOOL for discrete states/commands and REAL for tank-level values and setpoints.

Step 4. Set the operating values: TankEmpty_SP = 2.0 ft, TankFull_SP = 8.0 ft, TankLoAlarm_SP = 1.0 ft and TankHiAlarm_SP = 9.0 ft.

Step 5. Insert left and right power rails with enough pins for the complete program, then create the first logic rung.

Step 6. Build the valve-control rung: TankFillAuto_Cmd AND NOT TankLevelHi_Alarm energizes TankValveOpen.

Step 7. Build the automatic-start rung: when TankLevel_LIT <= TankEmpty_SP, SET TankFillAuto_Cmd.

Step 8. Build the automatic-stop/safety rung: when TankLevel_LIT >= TankFull_SP OR Estop_Cmd is TRUE OR TankLevelHi_Alarm is TRUE, RESET TankFillAuto_Cmd.

Step 9. Build the high-level alarm rung: TankLevel_LIT >= TankHiAlarm_SP energizes TankLevelHi_Alarm.

Step 10. Build the low-level alarm rung: TankLevel_LIT <= TankLoAlarm_SP energizes TankLevelLo_Alarm.

Step 11. Check every variable name and data type. Confirm that all contacts/coils/expressions are wired to complete rungs and that no instruction is floating.

Step 12. Save the project. If the editor provides validation/compile checking in your VM image, run it and correct any errors before submission.

Step 13. Create a logic test table covering at least six scenarios: empty tank, normal filling, full tank, high alarm, low alarm and emergency stop.

Step 14. Capture screenshots of the variable table and the completed ladder logic. Submit the project folder together with your written report.

Required Evidence & Submission

- Screenshot of the completed variable/tag table.
- Screenshot(s) of the complete ladder program.
- A six-scenario logic test table showing inputs/conditions and expected outputs.
- The exported/saved TankFill OpenPLC project files.
- A short safety explanation describing why an emergency-stop condition must override normal automatic control.

Recommended filename: <RegistrationNo>_LAB03_Building_Your_First_PLC_Program.pdf plus the required project/capture file(s).

Solution:

1. Objective and Environment

The objective of this lab was to build a Ladder Diagram program in OpenPLC Editor that automatically fills a 0-10 ft tank between two setpoints, raises high and low level alarms, and stops filling when an emergency stop or high-level condition occurs. The work was done in the ICDFA PLC virtual machine, in a new project folder named TankFill containing a program (POU) named TankFill written in LD.

2. Variable Table

Ten variables were created, all with Class Local. BOOL is used for discrete states and commands, and REAL for the tank level and setpoints. The four setpoints were entered as initial values (2.0, 8.0, 1.0 and 9.0 ft), and TankLevel_LIT was given an initial value of 5.0 ft for testing.

#	Name	Class	Type	Location	Initial Value	Option	Documentation
1	TankFillAuto_Cmd	Local	BOOL				Automatic fill latch/com
2	TankValveOpen	Local	BOOL				Fill valve open comman
3	Estop_Cmd	Local	BOOL				Emergency-stop input
4	TankLevelHi_Alarm	Local	BOOL				High-level alarm
5	TankLevelLo_Alarm	Local	BOOL				Low-level alarm
6	TankLevel_UT	Local	REAL		5.0		Tank level in feet
7	TankEmpty_SP	Local	REAL		2.0		Fill start setpoint
8	TankFull_SP	Local	REAL		8.0		Fill stop setpoint
9	TankLoAlarm_SP	Local	REAL		1.0		Low alarm setpoint
10	TankHiAlarm_SP	Local	REAL		9.0		High alarm setpoint

Figure 1: TankFill variable table in OpenPLC Editor (10 variables, BOOL and REAL).

2.1 Purpose of each variable

Variable	Type	Initial value	Purpose
TankFillAuto_Cmd	BOOL	-	Internal automatic fill latch/command
TankValveOpen	BOOL	-	Discrete command for the fill valve
Estop_Cmd	BOOL	-	Emergency-stop input; TRUE forces automatic fill to stop
TankLevelHi_Alarm	BOOL	-	High-level alarm state
TankLevelLo_Alarm	BOOL	-	Low-level alarm state
TankLevel_LIT	REAL	5.0	Measured tank level in feet
TankEmpty_SP	REAL	2.0	Automatic fill start setpoint (ft)
TankFull_SP	REAL	8.0	Automatic fill stop setpoint (ft)
TankLoAlarm_SP	REAL	1.0	Low-level alarm setpoint (ft)
TankHiAlarm_SP	REAL	9.0	High-level alarm setpoint (ft)

3. Ladder Logic Program

The program has five functional rungs built between a left and a right power rail with 10 pins each. Comparisons are written as variable expressions, and the latch is implemented with a SET coil and a RESET coil on TankFillAuto_Cmd.

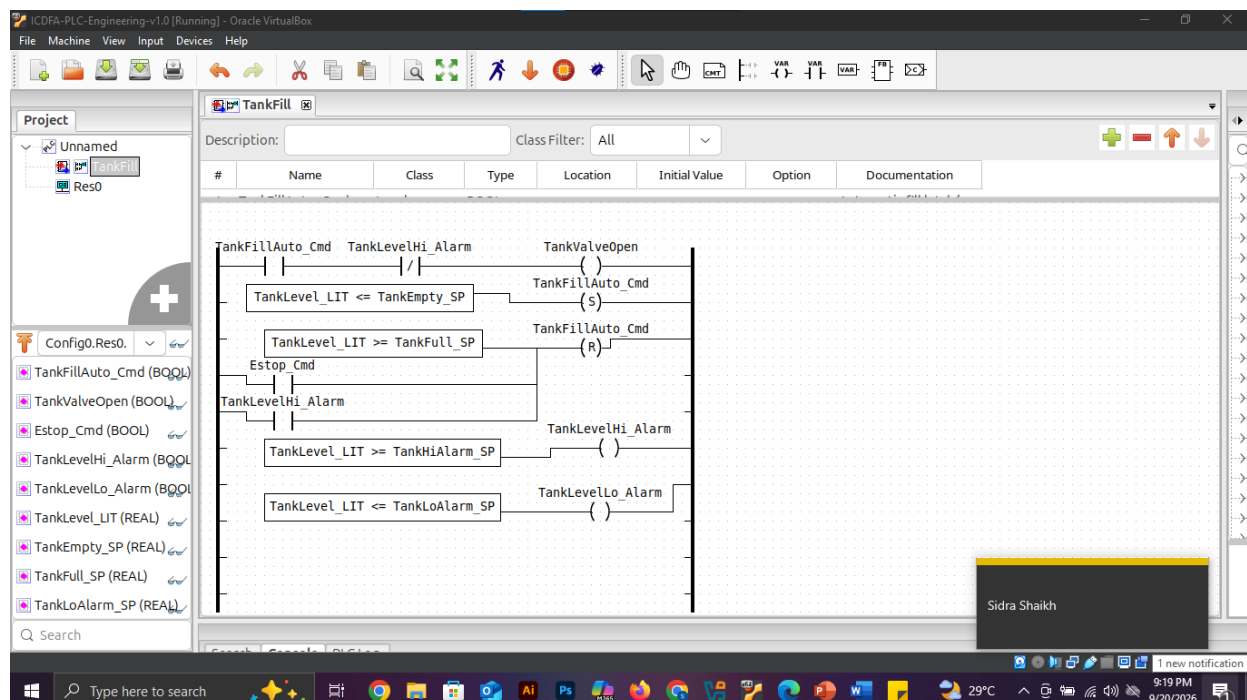


Figure 2: Complete TankFill ladder program in OpenPLC Editor (full window).

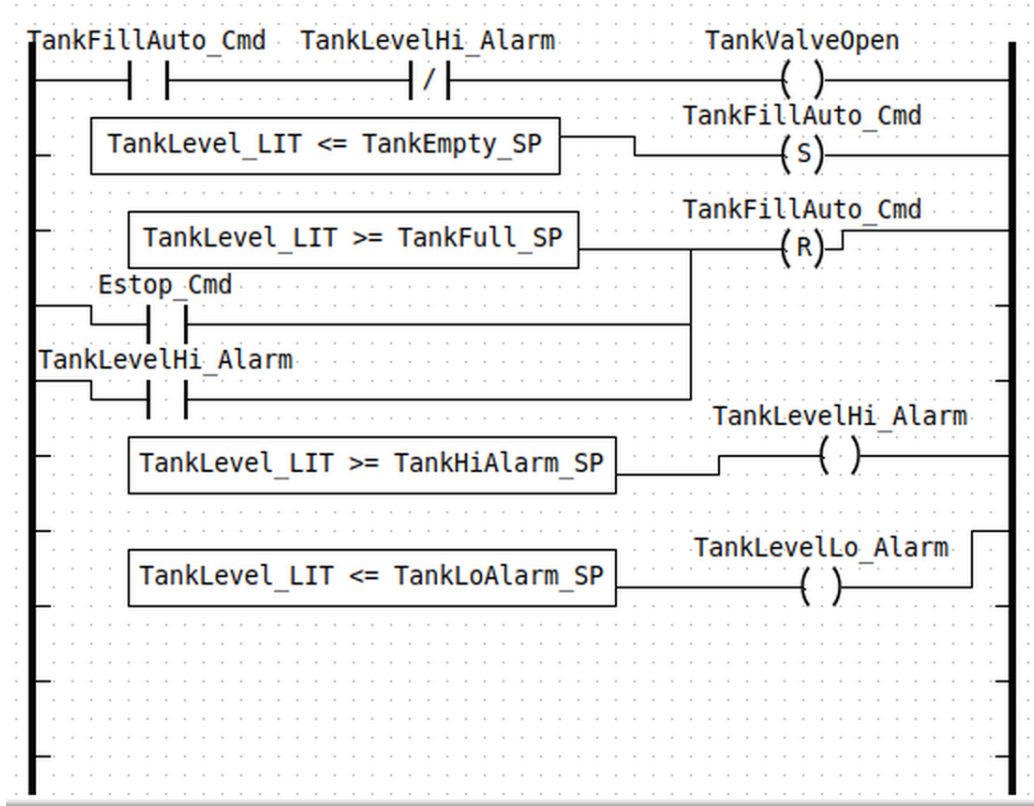


Figure 3: Enlarged view of the same ladder program for readability.

3.1 Rung-by-rung explanation

Rung	Logic	Purpose
1	TankFillAuto_Cmd (normal contact) in series with TankLevelHi_Alarm (negated contact) energizes the TankValveOpen coil.	The valve opens only while the automatic fill command is on and the high-level alarm is off.
2	TankLevel_LIT <= TankEmpty_SP sets TankFillAuto_Cmd (SET coil).	Starts an automatic fill when the level falls to 2.0 ft or below. The command stays latched afterwards.
3	TankLevel_LIT >= TankFull_SP OR Estop_Cmd OR TankLevelHi_Alarm resets TankFillAuto_Cmd (RESET coil). The three conditions are parallel branches.	Stops filling at 8.0 ft, on an emergency stop, or on a high-level alarm.
4	TankLevel_LIT >= TankHiAlarm_SP energizes TankLevelHi_Alarm.	High-level alarm at 9.0 ft, for when the valve fails to close.
5	TankLevel_LIT <= TankLoAlarm_SP energizes TankLevelLo_Alarm.	Low-level alarm at 1.0 ft, for when the valve fails to open.

The RESET rung is placed below the SET rung. The PLC evaluates rungs in order and the last write wins, so when an emergency stop or high alarm is active at the same time as a low level, the reset overrides the automatic start.

3.2 Checks and validation

Every variable name and data type was checked against the Variable Plan, and all contacts, coils and expressions were wired to complete rungs with nothing left floating. The project was then saved and built in OpenPLC Editor to check the program.

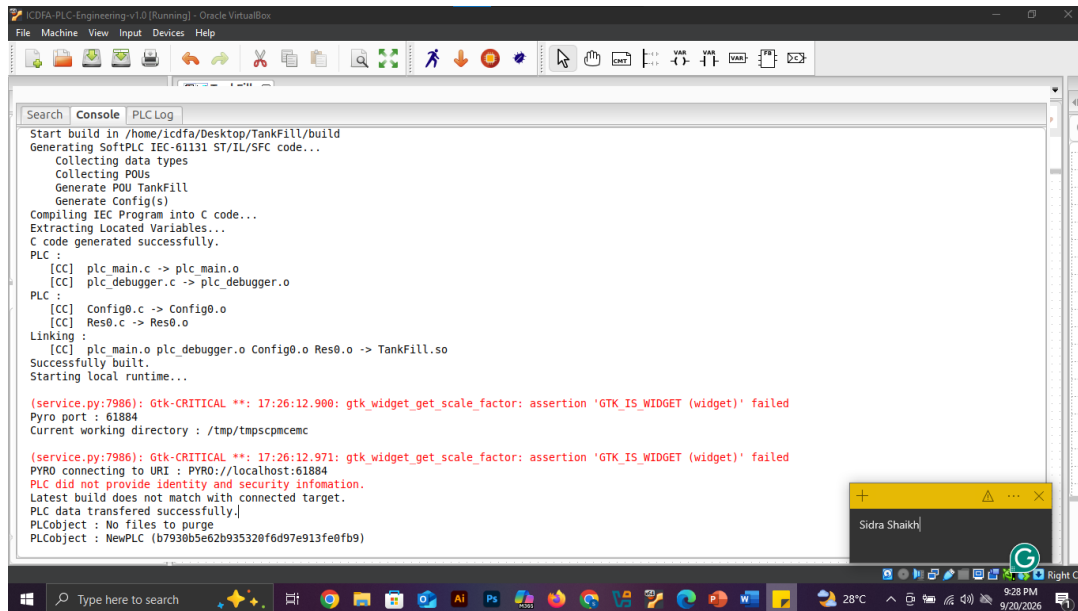


Figure 4: OpenPLC Editor Console showing the TankFill build finishing with Successfully built.

The Console shows the editor collecting the data types and POU, generating the TankFill POU and the configuration, generating the IEC 61131 code as C code, compiling the C files and linking them into TankFill.so. The build ended with Successfully built, and the program was then transferred to the local runtime. The GTK-CRITICAL messages and the note that the PLC did not provide identity information come from the editor interface and the local simulation runtime. They are not errors in the ladder program, and the build and transfer still completed.

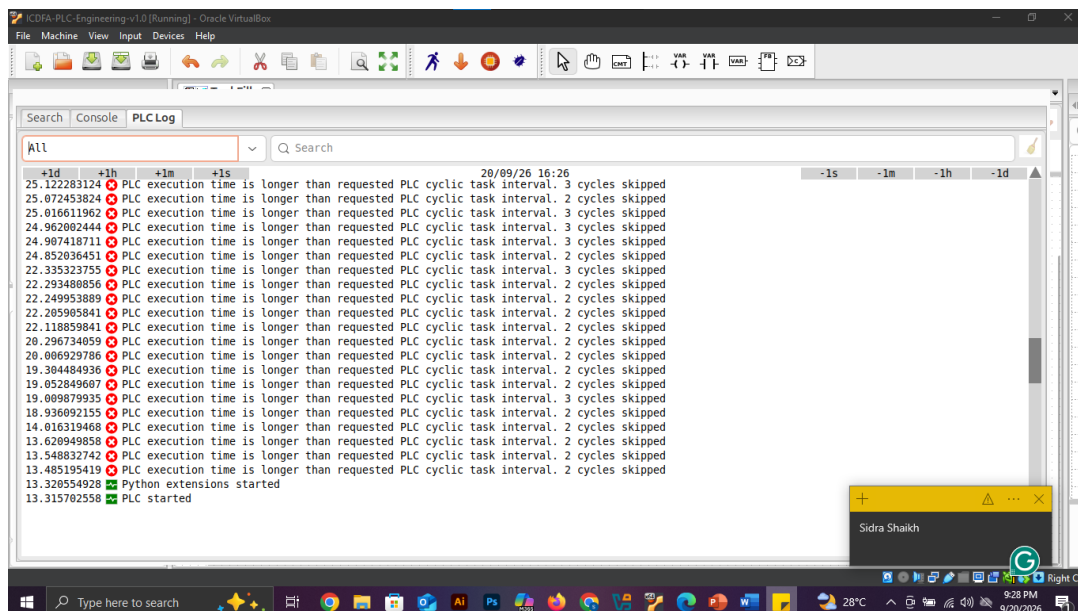


Figure 5: PLC Log showing the PLC started, followed by cycle-time warnings.

The PLC Log confirms that the PLC and the Python extensions started. It then shows repeated warnings that the execution time was longer than the requested cyclic task interval, with two or three cycles skipped. This is a timing warning, most likely because the virtual machine has limited processing speed compared with the configured task interval. It does not point to an error in the program logic, which built without errors.

4. Logic Test Table

The table lists seven scenarios (the lab requires six). The setpoints are 2.0 ft (fill start), 8.0 ft (fill stop), 1.0 ft (low alarm) and 9.0 ft (high alarm). Expected outputs are the settled values after one or two scans.

#	Scenario	Level (ft)	E-stop input	Latch before	Latch after	TankValveOpen	Hi alarm	Lo alarm
1	Empty tank (fill start) Level <= 2.0, so the SET rung latches the command.	2.0	FALSE	FALSE	TRUE	TRUE	FALSE	FALSE
2	Normal filling Between setpoints, no SET or RESET condition, so the latch holds.	5.0	FALSE	TRUE	TRUE (held)	TRUE	FALSE	FALSE
3	Full tank Level >= 8.0, so the RESET rung clears the latch.	8.0	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE
4	High alarm Level >= 9.0: alarm on, latch reset, and the negated alarm contact closes the valve.	9.2	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE
5	Low alarm Level <= 1.0: low alarm on, and the fill starts because the level is below 2.0.	0.8	FALSE	FALSE	TRUE	TRUE	FALSE	TRUE
6	Emergency stop while filling Estop_Cmd TRUE resets the latch and the valve closes.	5.0	TRUE	TRUE	FALSE	FALSE	FALSE	FALSE
7	E-stop with low level (priority test) The SET rung fires, but the RESET rung below it overrides in the same scan.	1.5	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE

5. Safety Explanation

An emergency-stop condition must override normal automatic control because its purpose is to protect people, equipment and the process when something is wrong. The automatic logic only follows level setpoints, so it cannot know about a burst pipe, an overflow risk or a hazard to an operator. In this program Estop_Cmd is one of

the conditions of the RESET rung, which is evaluated after the automatic-start SET rung. Even when the level is low enough to request a fill, the fill command is cleared and the valve closes (see test scenario 7). If normal control could override the emergency stop, the system could continue filling during a dangerous condition.

6. Project Files

The project was saved in the folder TankFill (/home/icdfa/Desktop/TankFill in the VM). It contains the OpenPLC project files, including plc.xml, and the build output generated when the program was compiled. The project folder is submitted together with this report in the github repository link:

https://github.com/SidraShaikh-2/ICDFA-Assignments/tree/main/ACICP301%20%E2%80%94%20ICS%20%26%20SCADA%20Systems/C11-26-ACIS-17349_LAB03_TankFill_ProjectFiles/TankFill

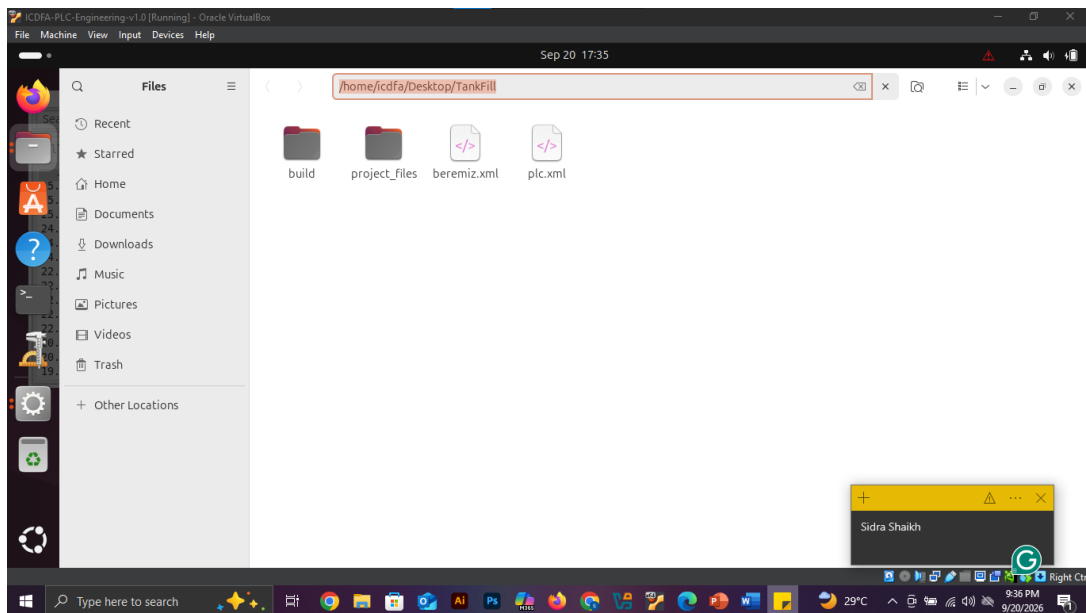


Figure 6: Saved folder.

7. Knowledge Check

1. Why are BOOL and REAL appropriate data types for this project?

Response: BOOL suits two-state items such as the valve command, the emergency stop and the alarms, which are either ON or OFF. REAL suits the tank level and the setpoints, which are continuous values with decimals such as 7.5 ft, and which whole-number types cannot represent accurately.

2. What is the purpose of a SET/RESET pair for TankFillAuto_Cmd?

Response: The pair works as a latch, or memory. When the level drops to the start setpoint the SET coil turns the command on, and it stays on while the tank fills, even though the level has risen above 2.0 ft. It is turned off only when a RESET condition occurs. This gives two distinct setpoints, so the valve does not open and close repeatedly around a single level.

3. What should happen if the tank reaches 9 ft while the fill valve remains commanded open?

Response: The high-level alarm should turn on to alert the operator, the RESET rung should clear the fill command, and the valve rung, which has a negated high-alarm contact, should close the valve immediately. In a real plant an independent hardwired overflow protection should also be present in case the PLC or valve fails.

4. Why should PLC logic be validated in simulation before use with physical equipment?

Response: Errors in the logic, such as a wrong setpoint, a missing interlock or a wrong rung order, could overfill a tank, damage equipment or endanger people. Validating the behaviour in simulation lets these faults be found and corrected safely before the program controls real hardware.

8. Conclusion

A complete automatic tank-fill program was built in Ladder Diagram using Boolean and Real variables, a SET/RESET latch, high and low alarms, and an emergency-stop override. The logic test table shows the expected behaviour for normal, alarm and emergency conditions, and the placement of the RESET rung guarantees that safety conditions take priority over automatic starting.

Marking Rubric — 100 Marks

Criterion	Marks	Standard
Technical execution	50	Lab procedure completed correctly; required configuration/analysis works as intended.
Evidence	20	Screenshots/files are complete, readable, relevant and clearly labelled.
Analysis & knowledge check	20	Responses demonstrate understanding of the industrial-control concepts observed.
Professional reporting	10	Clear structure, correct terminology, concise writing, file naming and submission compliance.

Submission Integrity: Evidence must come from the student's own assigned laboratory environment. Screenshots should show enough context to verify the activity. Do not submit another learner's screenshots or project files.